

Izveštaj o performansama

REST vs GraphQL vs gRPC

Komparativna analiza mikroservisnih protokola za IoT sistem

Alat za testiranje: k6 | Kontejnerizacija: Docker Compose

Baza podataka: PostgreSQL (IoT optimizovana)

1. Uvod

Ovaj izveštaj predstavlja rezultate komparativne analize performansi tri komunikaciona protokola (REST, GraphQL i gRPC) implementiranih kao odvojeni mikroservisi koji pristupaju zajedničkoj PostgreSQL bazi podataka optimizovanoj za IoT podatke. Baza koristi indeksiranje po vremenu i ID-u uređaja radi efikasnog pristupa vremenskim serijama senzorskih podataka.

Sistem je u potpunosti kontejnerizovan pomoću Docker Compose-a, a testiranje je sprovedeno korišćenjem alata k6 sa tri specifična IoT scenarija koji pokrivaju različite aspekte komunikacije: masovni upis podataka, selektivno čitanje i složene analitičke upite.

1.1 Tehnologije i arhitektura

Svaki od tri servisa implementiran je kao zaseban kontejner sa sledećim karakteristikama:

REST servis koristi standardne HTTP endpointe sa JSON formatom i OpenAPI (Swagger) dokumentacijom. Predstavlja najrasprostranjeniji pristup API dizajnu sa jasnom semantikom HTTP metoda.

GraphQL servis omogućava klijentu selektovanje specifičnih polja čime se izbegava over-fetching podataka. Koristi jedinstven endpoint sa fleksibilnim upitnim jezikom.

gRPC servis koristi binarnu komunikaciju putem Protocol Buffers (.proto definicije) što obezbeđuje efikasnu serijalizaciju i manju veličinu poruka u poređenju sa tekstualnim formatima.

1.2 Metodologija merenja

Load testovi su izvršeni sa k6 alatom uz postepeno skaliranje do 500 virtuelnih korisnika (VU). Tokom testiranja praćeni su: prosečna latencija (avg), percentilne latencije (p90, p95), broj zahteva u sekundi (RPS), stopa uspešnosti (check rate), kao i zauzeće CPU i RAM resursa svakog kontejnera putem komande docker stats sa intervalom uzorkovanja od 2 sekunde.

2. Scenario A – High-Frequency Ingestion

Ovaj scenario simulira IoT uređaje koji šalju podatke u kratkim intervalima. Fokus je na brzini upisa (write-heavy workload) i overhead-u protokola pri kreiranju novih zapisa. Svi servisi su testirali operaciju kreiranja novog senzorskog zapisa.

2.1 Rezultati latencije

Metrika	REST	GraphQL	gRPC	Prag
Avg latencija	7.64 ms	14.82 ms	11.1 ms	avg < 1000 ms ✓
p95 latencija	29.1 ms	38.79 ms	48.25 ms	p95 < 2000 ms ✓
p90 latencija	14.92 ms	16.63 ms	26.42 ms	—
Medijana	3.12 ms	3.95 ms	3.09 ms	—
Max latencija	497 ms	16.4 s	301 ms	—

2.2 Propusnost i pouzdanost

Metrika	Vrednost
Ukupan RPS (HTTP)	116.19 req/s
Iteracije/s	56.75 iter/s
Ukupno provera	44,134 (99.85% uspešnih)
HTTP neuspešni	0.00% (0 od 22,400)
gRPC neuspešni	0.59% (62 od 10,534)
Max VU	500

2.3 Resursi (CPU i RAM)

Analiza docker stats podataka tokom scenarija A pokazuje sledeće obrasce zauzeća resursa:

Resurs	REST	GraphQL	gRPC	DB
Peak CPU	35.14%	108.17%	105.28%	182.86%
Avg CPU (load)	~10–20%	~15–30%	~15–40%	~20–80%
RAM (stabilno)	~101 MiB	~210–265 MiB	~657–703 MiB	~44–155 MiB
RAM %	~1.3%	~2.7–3.4%	~8.4–9.0%	~0.6–2.0%

2.4 Zaključak – Scenario A

Scenario A je uspešno izvršen sa svim pragovima zadovoljenim. REST je pokazao najbolje performanse sa najnižom prosečnom latencijom od 7.64 ms, što ga čini najefikasnijim za jednostavne operacije upisa. GraphQL je bio nešto sporiji (14.82 ms) zbog overhead-a parsiranja

upita, dok je gRPC imao prosečnu latenciju od 11.1 ms, ali sa najvišom p95 vrednošću (48.25 ms) što ukazuje na povremene spike-ove pri većem opterećenju.

Sa aspekta resursa, REST servis je bio najštedljiviji (peak CPU 35%, RAM ~101 MiB). gRPC servis zauzima značajno više memorije (~657–703 MiB) što je karakteristika Go/Java runtime-a i Protocol Buffers biblioteka. Baza podataka je bila usko grlo pri peak opterećenju (CPU do 182.86%), što ukazuje na potrebu za optimizacijom upisa ili dodavanjem write-ahead log konfiguracije.

Ukupna stopa uspešnosti od 99.85% potvrđuje stabilnost sistema pod high-frequency ingestion opterećenjem sa 500 VU.

3. Scenario B – Selective Monitoring

Scenario B simulira klijenta (npr. mobilnu aplikaciju) sa lošom vezom koji traži samo 2 od 10 dostupnih senzorskih vrednosti. Ovaj scenario direktno testira prednost GraphQL-ovog selektivnog čitanja podataka naspram REST-ovog fiksno odgovora i gRPC-ove binarne komunikacije.

3.1 Rezultati latencije

Metrika	REST	GraphQL	gRPC	Status
Avg latencija	6.16 ms	20.64 s	54.21 ms	REST/gRPC ✓
p95 latencija	21.78 ms	1m 0s	194.33 ms	GQL ✗ FAIL
Medijana	2.73 ms	11 s	41.27 ms	—
Max latencija	340 ms	1m 0s	302.6 ms	—
HTTP fail rate	0%	9.19%	0%	—

3.2 Propusnost i pouzdanost

Metrika	Vrednost
Check rate	73.32% — FAIL (prag > 95%)
GraphQL 200 OK	79% (965 / 1210)
GraphQL no errors	13% (164 / 1210)
REST 200 OK	100% ✓
gRPC StatusOK	100% ✓
Data received	842 MB (4.4 MB/s)
Iteracije	1,210 (245 prekinutih)

3.3 Resursi (CPU i RAM)

Resurs	REST	GraphQL	gRPC	DB
Peak CPU	46.43%	338.87%	68.48%	231.06%
Avg CPU (load)	~1–10%	~110–135%	~0–5%	~10–40%
RAM (peak)	~116 MiB	~1.37 GiB	~756 MiB	~170 MiB
RAM %	~1.5%	~17–18%	~9–10%	~1.5–2.2%

3.4 Zaključak – Scenario B

Scenario B je otkrio kritičan problem sa GraphQL servisom. Uprkos tome što je GraphQL dizajniran upravo za ovakve scenarije selektivnog čitanja, servis je pod opterećenjem od 500 VU doživeo ozbiljan degradaciju performansi sa prosečnom latencijom od 20.64 sekundi i p95 od

punog minuta (timeout). Samo 13% GraphQL zahteva je prošlo bez grešaka, a CPU zauzeće je konstantno bilo iznad 100% (peak 338.87%).

Uzrok ovog ponašanja je verovatno nedostatak optimizacije na nivou resolvera, problem N+1 upita prema bazi, ili nedovoljno konfigurisano connection pooling. GraphQL servis je primio čak 842 MB podataka što ukazuje na potencijalno neadekvatnu implementaciju upita ili greške u resolver logici.

Sa druge strane, REST i gRPC su pokazali odlične rezultate. REST je ponovo bio najbrži (avg 6.16 ms) sa 100% uspešnošću, dok je gRPC takođe zadržao potpunu pouzdanost sa prosečnom latencijom od 54.21 ms. gRPC je pritom bio izuzetno efikasan po pitanju CPU resursa (većinu vremena blizu 0%), što dokazuje efikasnost binarne Protobuf serijalizacije.

Baza podataka je doživela peak CPU od 231% što je direktna posledica velikog broja konkurentnih GraphQL upita koji su generisali intenzivan I/O opterećenje.

4. Scenario C – Heavy Querying

Scenario C predstavlja najzahtevniji test sa složenim upitima nad velikim opsegom istorijskih podataka uključujući agregacije (npr. prosečna temperatura). Ovaj scenario simulira analitičke zahteve tipične za IoT monitoring dashboarde.

4.1 Rezultati latencije

Metrika	REST	GraphQL	gRPC	Status
Avg latencija	11.37 ms	1m 0s	9.44 ms	GQL x FAIL
p95 latencija	14.7 ms	1m 0s	12.07 ms	GQL x FAIL
Medijana	9.68 ms	1m 0s	8.84 ms	—
Max latencija	343 ms	1m 0s	31.36 ms	—
HTTP fail rate	0%	37.26%	0%	—

4.2 Propusnost i pouzdanost

Metrika	Vrednost
Check rate	50.00% — FAIL (prag > 95%)
GraphQL 200 OK	0% (0 / 411) — Potpun otkaz
REST 200 OK	100% ✓
REST has AvgTemp	0% (0 / 692) — Nedostaje polje
gRPC StatusOK	100% ✓
gRPC has avgTemp	100% ✓
Iteracije	406 (avg trajanje: 1m 0s)

4.3 Resursi (CPU i RAM)

Resurs	REST	GraphQL	gRPC	DB
Peak CPU	167.59%	250.19%	54.89%	794.04%
Avg CPU (load)	~20–110%	~108–165%	~15–55%	~620–795%
RAM (peak)	~198 MiB	~1.40 GiB	~795 MiB	~248 MiB
RAM %	~2.5%	~18%	~10%	~3.1%

4.4 Zaključak – Scenario C

Scenario C je definitivno najkritičniji i otkrio je fundamentalne probleme u implementaciji. GraphQL servis je potpuno otkazao – nijedan zahtev nije uspešno vraćen (0% success rate za 200 OK i 0% za no errors). Svi GraphQL zahtevi su završavali sa timeout-om od 60 sekundi. CPU zauzeće GraphQL servisa je konstantno bilo iznad 100%, a memorija je dostigla 1.40 GiB.

Posebno zabrinjavajuć je podatak da je baza podataka dostigla peak CPU od čak 794.04% što ukazuje na izuzetno neoptimizovane upite koji generišu full table scan operacije bez odgovarajućih indeksa za agregacione upite. Ovo je direktna posledica GraphQL resolvera koji verovatno za svako polje vrše poseban upit (N+1 problem) ili koriste neoptimizovane SQL agregacije.

gRPC je pokazao izvanredne rezultate u ovom scenariju sa prosečnom latencijom od samo 9.44 ms i 100% uspešnošću na svim proverama uključujući avgTemp agregaciju. Ovo dokazuje da je gRPC implementacija imala pravilno optimizovane upite i efikasnu serijalizaciju agregatnih rezultata.

REST je imao dobre latencije (avg 11.37 ms) i 100% HTTP uspešnost, ali je provera "has AvgTemp" pala na 0%, što znači da REST endpoint za agregaciju nije implementiran ili ne vraća očekivano polje u odgovoru. REST peak CPU od 167.59% ukazuje na značajno opterećenje tokom agregacionih upita.

5. Komparativna analiza

5.1 Pregled po protokolima

Aspekt	REST	GraphQL	gRPC
Scen. A	✓ Najbolji (7.64 ms)	✓ OK (14.82 ms)	✓ Dobar (11.1 ms)
Scen. B	✓ Odličan (6.16 ms)	✗ Otkazao (20.6 s)	✓ Dobar (54.21 ms)
Scen. C	~ Delimično *	✗ Potpun otkaz	✓ Najbolji (9.44 ms)
Peak CPU	167.59%	338.87%	105.28%
RAM footprint	~101 MiB	~265–1.40 GiB	~657–795 MiB

* REST u scenariju C ima dobre latencije ali ne vraća agregaciono polje (AvgTemp), što ukazuje na nepotpunu implementaciju endpoint-a za agregaciju.

5.2 REST – Ukupna ocena

REST se pokazao kao najstabilniji i najkonzistentniji protokol kroz sva tri scenarija. Sa najnižim latencijama u scenarijima A (7.64 ms) i B (6.16 ms), REST demonstrira izuzetnu efikasnost za jednostavne CRUD operacije. Minimalan RAM footprint (~101 MiB) i predvidljiv CPU profil čine ga idealnim za resurski ograničene okoline. Jedina slabost je nepotpuna implementacija agregacionog endpoint-a u scenariju C, što nije inherentan problem protokola već implementacije.

5.3 GraphQL – Ukupna ocena

GraphQL je pokazao zadovoljavajuće rezultate jedino u scenariju A gde je opterećenje bilo ravnomerno raspoređeno. U scenarijima B i C, servis je doživeo ozbiljne probleme sa performansama – od prosečne latencije od 20 sekundi (scenario B) do potpunog otkaza sa 0% uspešnošću (scenario C). Konstantno CPU zauzeće iznad 100% i memorija do 1.40 GiB ukazuju na fundamentalne probleme u implementaciji: verovatno N+1 problem sa upitima, nedostatak DataLoader mehanizma, neadekvatno connection pooling, ili neoptimizovani resolveri za agregacione upite.

Ironično, scenario B (selektivno čitanje) bi teoretski trebalo da bude GraphQL-ova najjača strana jer omogućava traženje samo potrebnih polja. Loši rezultati u ovom scenariju jasno ukazuju da problem nije u samom protokolu već u kvalitetu implementacije.

5.4 gRPC – Ukupna ocena

gRPC je pokazao izuzetnu pouzdanost sa 100% uspešnošću na svim kritičnim proverama u sva tri scenarija (jedini koji je imao potpunu funkcionalnost u scenariju C uključujući agregaciju). Posebno se istakao u scenariju C sa prosečnom latencijom od samo 9.44 ms za složene agregacione upite, što je impresivan rezultat koji potvrđuje efikasnost binarne Protobuf serijalizacije i dobro optimizovanih server-side procedura.

Glavni nedostatak gRPC-a je značajno veći RAM footprint (657–795 MiB) u poređenju sa REST-om, što je cena runtime okruženja i Protocol Buffers biblioteka. Međutim, CPU efikasnost je

odlična – u scenariju B, gRPC je većinu vremena imao CPU blizu 0% dok je obrađivao zahteve, što dokazuje minimalnu cenu binarne serijalizacije.

6. Analiza resursa i cena serijalizacije

Jedan od ključnih aspekata ovog projekta je procena koliko "košta" serijalizacija i deserijalizacija podataka na procesorskom nivou za svaki protokol.

6.1 CPU cena serijalizacije

REST (JSON): JSON serijalizacija je najlakša po CPU, što potvrđuje najniže CPU korišćenje REST servisa kroz sve scenarije. Tekstualni format je efikasno optimizovan u modernim runtime okruženjima i ne zahteva kompleksnu obradu.

gRPC (Protobuf): Binarna serijalizacija kroz Protocol Buffers je efikasnija od JSON-a po pitanju veličine poruka, ali sama operacija serijalizacije/deserijalizacije ne donosi značajnu CPU prednost u low-latency scenarijima. Veći baseline RAM je cena koja se plaća za strict typing i generisane stub-ove.

GraphQL (JSON + Query parsing): GraphQL ima dodatni overhead parsiranja upita, validacije sheme i resolver izvršavanja pre same JSON serijalizacije. Ovo objašnjava konstantno višu CPU potrošnju i sporije odgovore – svaki zahtev zahteva analizu query stringa, matching sa shemom i višestruko pozivanje resolvera.

6.2 RAM profil

Analiza memorijskog korišćenja otkriva značajne razlike: REST servis održava stabilan footprint od ~101 MiB nezavisno od opterećenja, GraphQL servis raste od ~210 MiB (idle) do 1.40 GiB (peak u scenariju C) što ukazuje na memory leak ili neoptimizovano keširanje, dok gRPC servis ima viši baseline (~657 MiB) ali relativno stabilan rast.

6.3 Baza podataka kao usko grlo

PostgreSQL baza je dostigla alarmantne CPU vrednosti: 182.86% u scenariju A, 231.06% u scenariju B i neverovatnih 794.04% u scenariju C. Ovaj eksponencijalni rast ukazuje da baza nije bila adekvatno pripremljena za agregacione upite. Preporuke uključuju: kreiranje materialized views za česte agregacije, dodavanje composite indeksa za vremenske opsege sa device_id, konfiguraciju work_mem i shared_buffers parametara, kao i implementaciju connection poolinga (npr. PgBouncer).

7. Generalni zaključak

Na osnovu sprovedene analize tri IoT scenarija sa tri komunikaciona protokola, mogu se izvesti sledeći ključni zaključci:

REST se pokazao kao najstabilniji, najbrži i najresursno efikasniji protokol za većinu IoT use case-ova. Sa konzistentno niskim latencijama (6–12 ms), minimalnim RAM footprint-om i 100% HTTP uspešnošću u svim scenarijima, REST predstavlja siguran izbor za IoT sisteme gde je jednostavnost i pouzdanost prioritet.

gRPC je jedini protokol koji je uspešno prošao sve funkcionalne provere u sva tri scenarija, uključujući agregacione upite u scenariju C. Binarna Protobuf komunikacija obezbeđuje manje poruke i efikasniju serijalizaciju, što ga čini idealnim za machine-to-machine komunikaciju u IoT okruženjima gde je propusni opseg ograničen.

GraphQL nije ispunio očekivanja u ovom testu. Iako protokol sam po sebi nudi značajne prednosti (selektivno čitanje, izbegavanje over-fetching-a), implementacija nije bila optimizovana za visoko opterećenje. Rezultati u scenarijima B i C (prosečna latencija od 20 sekundi do potpunog timeout-a) jasno pokazuju da GraphQL zahteva znatno više pažnje pri implementaciji: DataLoader pattern za batch-ovanje upita, pravilno keširanje, optimizovane resolvere i adekvatno connection pooling.

Konačno, analiza je pokazala da je PostgreSQL baza podataka bila značajno usko grlo, posebno u scenariju C sa CPU do 794%. Za produkcijske IoT sisteme preporučuje se korišćenje specijalizovanih time-series baza (npr. TimescaleDB ekstenzija za PostgreSQL) ili implementacija materialized views za agregacione upite.